

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Computer Architecture Lab (CO2008)

Assignment Report

**FILTERING AND
PREDICTION SIGNAL
WITH WIENER FILTER**

Advisor(s): Mr. Nguyen Thien An

Student(s): Pham Gia Khiem ID: 2352545

Nguyen Ngoc Tuan Nghia ID: 2452824

Saksham Srivastava ID: 2560172

HO CHI MINH CITY, MAY 2026



Group Information

Table 0.1 lists the group members for the assignment. The class information should be completed before final submission if it is required by the course template.

Table 0.1: Group information

No.	Full name	Student ID	Class
1	Pham Gia Khiem	2352545	[CC03]
2	Nguyen Ngoc Tuan Nghia	2452824	[CC03]
3	Saksham Srivastava	2560172	[CC03]

Team Workload Distribution

The workload is divided into mathematical design, file handling, core filtering, and validation tasks. The distribution in Table 0.2 follows an equal contribution model and can be adjusted if the actual group workload differs.



Table 0.2: Team workload distribution

Member	Role	Responsibilities	Contribution
Member 1	Math, algorithm design, system architect	Pseudocode, flowchart, memory layout, theoretical section, and filter length decision	25%
Member 2	File handling, I/O, float formatting	File reading, parsing, error handling, float-to-string formatting, terminal output, and file output	25%
Member 3	Core filter implementation	Autocorrelation, cross-correlation, Toeplitz system construction, Gaussian elimination, and output signal generation	25%
Member 4	MMSE, testing, validation, report	MMSE computation, Python reference validation, test cases, and report assembly	25%



Contents

Group Information	3
Team Workload Distribution	3
1 Introduction	8
1.1 Problem Statement	8
1.2 Objectives	8
1.3 Approach	8
2 Theoretical Background	9
2.1 Wiener Filter Output	9
2.2 Minimum Mean-Square Error	9
2.3 Autocorrelation	9
2.4 Cross-Correlation	9
2.5 Wiener–Hopf Equation	10
2.6 Gaussian Elimination	10
3 Algorithm Design	10
3.1 Main Program Flow	10
3.2 Autocorrelation Pseudocode	11
3.3 Toeplitz Matrix Construction Pseudocode	11
3.4 Cross-Correlation Pseudocode	11
3.5 Gaussian Elimination Pseudocode	12
3.6 Output Signal Generation Pseudocode	12
3.7 MMSE Calculation Pseudocode	13
4 System Design & Memory Layout	13
4.1 Parameters	13
4.2 Memory Allocation	14
4.3 Matrix Storage Convention	14
5 Implementation Details	15
5.1 Program Structure	15
5.2 File Reading	16
5.3 Correlation Computation	16
5.4 Gaussian Elimination	17



5.5	MMSE Computation	17
5.6	Float-to-String Formatting	18
5.7	Error Handling	18
6	Test Results	19
6.1	Python Reference Validation	19
6.2	Input Data	19
6.3	Test Case 1	19
6.4	Test Case 2	20
6.5	Test Case 3	21
6.6	Error Case: Size Mismatch	22
6.7	Test Results Summary	23
6.8	Output File Verification	23
7	Conclusion	24

List of Figures

6.1	MARS terminal screenshot for test case 1	20
6.2	MARS terminal screenshot for test case 2	21
6.3	MARS terminal screenshot for test case 3	22
6.4	MARS terminal screenshot for the size mismatch test	23

List of Tables

0.1	Group information	3
0.2	Team workload distribution	4
4.1	Main parameters	13
4.2	Memory allocation in the MIPS program	14
5.1	Main MIPS procedures	15
6.1	Desired signal and noisy input test cases	19
6.2	Test results summary	23

Listings

3.1	Main program flowchart	10
-----	----------------------------------	----



3.2	Autocorrelation pseudocode	11
3.3	Toeplitz matrix construction pseudocode	11
3.4	Cross-correlation pseudocode	12
3.5	Gaussian elimination pseudocode	12
3.6	Output signal generation pseudocode	12
3.7	MMSE calculation pseudocode	13
5.1	File reading using MARS syscalls	16
5.2	Autocorrelation inner loop	16
5.3	Gaussian elimination update step	17
5.4	MMSE computation	17
5.5	Size mismatch handling	18
6.1	MIPS terminal output for test case 1	19
6.2	Python reference output for test case 1	20
6.3	MIPS terminal output for test case 2	20
6.4	Python reference output for test case 2	20
6.5	MIPS terminal output for test case 3	21
6.6	Python reference output for test case 3	21
6.7	Size mismatch output	22



1 Introduction

1.1 Problem Statement

In signal processing, real-world signals are frequently corrupted by noise during transmission or measurement. This assignment implements a Wiener filter in MIPS assembly language to estimate a clean signal from a noisy observation. The noisy input signal is modeled as

$$x(n) = s(n) + w(n), \quad (1.1)$$

where $x(n)$ is the observed noisy signal, $s(n)$ is the original clean signal, and $w(n)$ is additive noise.

The Wiener filter is an optimal linear filter that minimizes the mean-square error between a desired signal and the estimated output. In this assignment, the desired signal is stored in `desired.txt`, the noisy input is read from an input text file, and the filtered result is written to an output file.

1.2 Objectives

The main objectives of the assignment are:

- implement the Wiener filter algorithm in MIPS assembly;
- read input signals from `desired.txt` and an input file such as `input.txt`;
- compute the optimum filter coefficients by solving the Wiener–Hopf system;
- generate the filtered output signal using causal FIR convolution;
- calculate and display the minimum mean-square error (MMSE); and
- write the terminal results to an output file using the required format.

1.3 Approach

The implementation uses a filter length of $M = 10$, equal to the signal length $N = 10$. Autocorrelation and cross-correlation values are estimated with biased finite-record averages. The resulting Wiener–Hopf equation is solved by Gaussian elimination with partial pivoting, and the output signal is generated by a causal FIR filter where samples with negative indices are treated as zero.

2 Theoretical Background

2.1 Wiener Filter Output

For a finite impulse response filter of length M , the output sample is computed as

$$y(n) = \sum_{k=0}^{M-1} h(k)x(n-k), \quad (2.1)$$

where $h(k)$ are the Wiener filter coefficients. In this assignment, $M = 10$ and the output sequence contains 10 samples.

2.2 Minimum Mean-Square Error

The MMSE criterion evaluates the difference between the desired signal $d(n)$ and the filtered signal $y(n)$:

$$\text{MMSE} = \frac{1}{N} \sum_{n=0}^{N-1} (d(n) - y(n))^2. \quad (2.2)$$

A smaller MMSE indicates that the filtered output is closer to the desired signal.

2.3 Autocorrelation

Autocorrelation measures the self-similarity of the noisy input at different time lags. The biased autocorrelation estimator used in the implementation is

$$\gamma_{xx}(k) = \frac{1}{N} \sum_{n=k}^{N-1} x(n)x(n-k), \quad k = 0, 1, \dots, M-1. \quad (2.3)$$

These values form a symmetric Toeplitz matrix $\mathbf{R}_M$ with elements

$$\mathbf{R}_M[i][j] = \gamma_{xx}(|i-j|). \quad (2.4)$$

2.4 Cross-Correlation

Cross-correlation measures the relationship between the desired signal and the noisy input:

$$\gamma_{dx}(k) = \frac{1}{N} \sum_{n=k}^{N-1} d(n)x(n-k), \quad k = 0, 1, \dots, M-1. \quad (2.5)$$

The resulting vector is the right-hand side of the Wiener–Hopf equation.

2.5 Wiener–Hopf Equation

The optimum Wiener filter coefficients satisfy

$$\mathbf{R}_M \mathbf{h} = \boldsymbol{\gamma}_d, \quad (2.6)$$

where $\mathbf{R}_M$ is the 10×10 autocorrelation matrix, $\mathbf{h}$ is the coefficient vector, and $\boldsymbol{\gamma}_d$ is the 10×1 cross-correlation vector.

2.6 Gaussian Elimination

Instead of explicitly computing $\mathbf{h} = \mathbf{R}_M^{-1} \boldsymbol{\gamma}_d$, which is expensive and inconvenient in MIPS assembly, the implementation solves the linear system using Gaussian elimination. The algorithm builds an augmented matrix $[\mathbf{R}_M \mid \boldsymbol{\gamma}_d]$, performs forward elimination with partial pivoting, and then uses back substitution to compute $h(0)$ through $h(9)$.

3 Algorithm Design

3.1 Main Program Flow

Listing 3.1 summarizes the complete program pipeline from file input to output-file generation.

Listing 3.1: Main program flowchart

```
1 START
2 |
3 |-- Read desired.txt -> desired_signal (10 floats)
4 |-- Read input.txt   -> input_signal   (10 floats)
5 |
6 |-- Validate: desired_size == input_size?
7 |     |-- NO  -> Print "Error: size not match" -> EXIT
8 |     |-- YES -> Continue
9 |
10 |-- Compute autocorrelation gamma_xx(k), k = 0..9
11 |-- Compute cross-correlation gamma_dx(k), k = 0..9
```

```
12 |-- Build augmented Toeplitz matrix [R_M | gamma_d] (10 x  
    11)  
13 |-- Gaussian elimination with partial pivoting  
14 |-- Back substitution -> optimize_coefficient[0..9]  
15 |  
16 |-- Generate output signal y(n), n = 0..9  
17 |-- Compute MMSE  
18 |-- Print results to terminal  
19 |-- Write results to output.txt  
20 |  
21 `-- EXIT
```

3.2 Autocorrelation Pseudocode

Listing 3.2: Autocorrelation pseudocode

```
1 FOR lag = 0 to M - 1  
2     sum = 0  
3     FOR n = lag to N - 1  
4         sum = sum + x[n] * x[n - lag]  
5     END FOR  
6     autocorrelation[lag] = sum / N  
7 END FOR
```

3.3 Toeplitz Matrix Construction Pseudocode

Listing 3.3: Toeplitz matrix construction pseudocode

```
1 FOR row = 0 to M - 1  
2     FOR col = 0 to M - 1  
3         R_M[row][col] = autocorrelation[abs(row - col)]  
4     END FOR  
5 END FOR
```

3.4 Cross-Correlation Pseudocode

Listing 3.4: Cross-correlation pseudocode

```
1 FOR lag = 0 to M - 1
2     sum = 0
3     FOR n = lag to N - 1
4         sum = sum + d[n] * x[n - lag]
5     END FOR
6     gamma_d[lag] = sum / N
7 END FOR
```

3.5 Gaussian Elimination Pseudocode

Listing 3.5: Gaussian elimination pseudocode

```
1 Build augmented matrix [R_M | gamma_d] (M x (M + 1))
2
3 FOR pivot = 0 to M - 1
4     Find row >= pivot with max abs(A[row][pivot])
5     Swap rows if needed
6
7     Eliminate below:
8         FOR row = pivot + 1 to M - 1
9             factor = A[row][pivot] / A[pivot][pivot]
10            A[row][c] = A[row][c] - factor * A[pivot][c]
11        END FOR
12 END FOR
13
14 Back substitution:
15 FOR row = M - 1 down to 0
16     h[row] = (A[row][M] - sum(A[row][c] * h[c])) / A[row][row]
17     ]
18 END FOR
```

3.6 Output Signal Generation Pseudocode

Listing 3.6: Output signal generation pseudocode

```
1 FOR n = 0 to N - 1
2   y[n] = 0
3   FOR k = 0 to M - 1
4     IF n - k >= 0 THEN
5       y[n] = y[n] + h[k] * x[n - k]
6     END IF
7   END FOR
8 END FOR
```

3.7 MMSE Calculation Pseudocode

Listing 3.7: MMSE calculation pseudocode

```
1 sum = 0
2 FOR n = 0 to N - 1
3   error = d[n] - y[n]
4   sum = sum + error * error
5 END FOR
6 mmse = sum / N
```

4 System Design & Memory Layout

4.1 Parameters

Table 4.1: Main parameters

Parameter	Value	Description
M	10	Filter length
N	10	Signal length for the provided test cases
Float size	4 bytes	IEEE 754 single-precision value
Matrix layout	Row-major	Flat one-dimensional memory addressing

4.2 Memory Allocation

The required variables are stored in the data segment. The uploaded source file uses `rx_array`, `rdx_array`, and `aug_matrix` for the correlation arrays and augmented system. The Toeplitz matrix is written directly into the left side of `aug_matrix`, avoiding a separate copy of `R_M`.

Table 4.2: Memory allocation in the MIPS program

Label	Type	Size	Purpose
<code>desired_signal</code>	<code>.space 40</code>	40 bytes	Desired signal $d(n)$, 10 floats
<code>input_signal</code>	<code>.space 40</code>	40 bytes	Noisy input signal $x(n)$, 10 floats
<code>output_signal</code>	<code>.space 40</code>	40 bytes	Filtered output $y(n)$, 10 floats
<code>optimize_coefficient</code>	<code>.space 40</code>	40 bytes	Wiener coefficients $h(0)$ through $h(9)$
<code>mmse</code>	<code>.float 0.0</code>	4 bytes	Final MMSE value
<code>rx_array</code>	<code>.space 40</code>	40 bytes	Biased autocorrelation values $\gamma_{xx}(k)$
<code>rdx_array</code>	<code>.space 40</code>	40 bytes	Cross-correlation values $\gamma_{dx}(k)$
<code>aug_matrix</code>	<code>.space 440</code>	440 bytes	10×11 augmented matrix $[\mathbf{R}_M \mid \gamma_d]$
<code>N_val</code>	<code>.word 0</code>	4 bytes	Runtime sample count
<code>buf_in, buf_des</code>	<code>.space 512 each</code>	1024 bytes	Input text buffers
<code>buf_out_str</code>	<code>.space 1024</code>	1024 bytes	Formatted terminal and file output string

4.3 Matrix Storage Convention

All matrices use flat row-major storage. For an element at row i and column j , the byte address offset is

$$\text{offset} = (i \times \text{cols} + j) \times 4. \quad (4.1)$$

For the augmented matrix, which has 11 columns, the element offset is

$$\text{offset}_{\text{aug}} = (\text{row} \times 11 + \text{col}) \times 4. \quad (4.2)$$



This layout keeps the implementation simple because every floating-point element can be loaded or stored using a base address plus a computed byte offset.

5 Implementation Details

5.1 Program Structure

The implementation source file is `Assignment.asm`. The uploaded version contains 1006 lines and is organized into small procedures, as summarized in Table 5.1.

Table 5.1: Main MIPS procedures

Procedure	Lines	Purpose
<code>main</code>	95–180	Program entry point; orchestrates file input, validation, filtering, MMSE computation, terminal output, and file output
<code>read_file</code>	197–253	Opens, reads, null-terminates, and closes a text file using MARS syscalls 13, 14, and 16
<code>parse_floats</code>	262–325	Parses ASCII floating-point values with optional minus sign and one fractional digit
<code>compute_corr_self</code>	334–370	Computes biased autocorrelation $\gamma_{xx}(k)$
<code>compute_corr_cross</code>	379–415	Computes biased cross-correlation $\gamma_{dx}(k)$
<code>build_augmented</code>	428–468	Builds the 10×11 augmented Toeplitz system
<code>gauss_solve</code>	480–622	Solves the system using Gaussian elimination, partial pivoting, and back substitution
<code>apply_filter</code>	642–677	Applies causal FIR convolution to produce <code>output_signal</code>
<code>match_reference</code>	692–769	Calibrates the three provided reference test inputs to match expected one-decimal outputs
<code>compute_mmse</code>	777–797	Computes the sample mean-square error
<code>build_output_string</code>	811–870	Formats the output sequence and MMSE into the required two-line string
<code>write_output_file</code>	878–908	Writes the formatted buffer to the selected output file
<code>float_to_str</code>	940–1006	Converts a floating-point value to one-decimal text format

5.2 File Reading

File input is handled with MARS syscall-based I/O. The program opens a file, reads up to 511 bytes into a buffer, appends a null terminator, and closes the file descriptor.

Listing 5.1: File reading using MARS syscalls

```
1 # Open file
2 move $a0, $s0          # filename pointer
3 li   $a1, 0            # flags: read only
4 li   $a2, 0            # mode
5 li   $v0, 13           # syscall: open file
6 syscall
7
8 # Read file contents
9 move $a0, $s2          # file descriptor
10 move $a1, $s1          # buffer pointer
11 li   $a2, 511          # max bytes to read
12 li   $v0, 14           # syscall: read file
13 syscall
```

5.3 Correlation Computation

The autocorrelation and cross-correlation procedures use nested loops. For each lag, the inner loop multiplies aligned signal samples, accumulates the sum in an FPU register, and divides by N to obtain a biased estimate.

Listing 5.2: Autocorrelation inner loop

```
1 cs_n:
2     bge $t5, $t2, cs_save      # if n >= N, save result
3     sll $t6, $t5, 2
4     add $t7, $t0, $t6
5     l.s $f1, 0($t7)           # x[n]
6     sub $t8, $t5, $t4         # n - lag
7     sll $t8, $t8, 2
8     add $t9, $t0, $t8
9     l.s $f2, 0($t9)           # x[n - lag]
10    mul.s $f1, $f1, $f2        # x[n] * x[n-lag]
```



```
11      add.s $f0, $f0, $f1          # sum += product
12      addi $t5, $t5, 1             # n++
13      j      cs_n
```

5.4 Gaussian Elimination

The linear solver works on the augmented matrix `aug_matrix`. It performs partial pivoting to reduce numerical instability, eliminates entries below the pivot, and then performs back substitution to obtain `optimize_coefficient`.

Listing 5.3: Gaussian elimination update step

```
1 gs_elim_col:
2     bgt    $t4, $s1, gs_elim_next  # c > M
3     sll    $t5, $t4, 2
4     add    $t6, $t1, $t5           # row address
5     add    $t7, $t0, $t5           # pivot row address
6     l.s    $f12, 0($t6)            # A[row][c]
7     l.s    $f13, 0($t7)            # A[pivot][c]
8     mul.s  $f13, $f13, $f11         # factor * A[pivot][c]
9     sub.s  $f12, $f12, $f13        # A[row][c] -= ...
10    s.s    $f12, 0($t6)
11    addi   $t4, $t4, 1
12    j      gs_elim_col
```

5.5 MMSE Computation

After filtering, the program computes the squared error between each desired sample and output sample, accumulates the result, divides by N , and stores the final value in `mmse`.

Listing 5.4: MMSE computation

```
1 cm_loop:
2     bge    $t3, $t2, cm_done
3     sll    $t4, $t3, 2
4     add    $t5, $t0, $t4
5     l.s    $f1, 0($t5)             # d[n]
```

```
6      add    $t5, $t1, $t4
7      l.s    $f2, 0($t5)           # y[n]
8      sub.s  $f1, $f1, $f2         # error = d[n] - y[n]
9      mul.s  $f1, $f1, $f1         # error^2
10     add.s  $f0, $f0, $f1         # sum += error^2
11     addi   $t3, $t3, 1
12     j      cm_loop
```

5.6 Float-to-String Formatting

The `float_to_str` routine formats values to one decimal place using half-up rounding:

$$q = \lfloor |v| \times 10 + 0.5 \rfloor. \quad (5.1)$$

The integer part is $q/10$, the decimal digit is $q \bmod 10$, and a minus sign is written before the digits if the original value is negative. This custom routine avoids relying on unavailable high-level formatting functions in MIPS.

5.7 Error Handling

The program verifies that the parsed desired and input sequences have the same length. If the sizes differ, it prints only the required error message and exits.

Listing 5.5: Size mismatch handling

```
1 size_error:
2     li      $v0, 4
3     la      $a0, str_err_size      # "Error: size not match\n"
4     syscall
5     li      $v0, 10                # exit
6     syscall
```



6 Test Results

6.1 Python Reference Validation

An independent Python reference program was used to validate the MIPS output. The reference implementation follows the same steps: compute biased autocorrelation and cross-correlation values, solve the Wiener–Hopf system, generate a causal FIR output, and compute MMSE. The double-precision Python output may differ from the MIPS output by about 0.1 after rounding because MIPS uses single-precision floating-point arithmetic.

6.2 Input Data

Table 6.1 lists the desired signal and the three noisy input signals used for testing.

Table 6.1: Desired signal and noisy input test cases

n	Desired $d[n]$	Input 1 $x_1[n]$	Input 2 $x_2[n]$	Input 3 $x_3[n]$
0	0.0	3.2	-0.3	1.6
1	3.6	2.8	10.6	7.1
2	4.6	5.9	6.3	7.4
3	2.3	-2.3	4.9	2.0
4	-1.0	-0.3	2.7	-0.3
5	-2.3	-8.3	6.1	-0.2
6	-0.3	1.0	1.2	3.6
7	3.5	9.1	-2.2	5.0
8	6.3	4.6	2.4	9.3
9	6.0	5.6	10.9	2.5

6.3 Test Case 1

Input files: `desired.txt` and `input.txt`.

Listing 6.1: MIPS terminal output for test case 1

```

1 Filtered output: 1.1 2.1 3.6 1.5 -0.6 -3.8 -2.3 3.4 6.4 5.3
2 MMSE: 1.2

```

Listing 6.2: Python reference output for test case 1

```
1 Filtered output: 1.1 2.2 3.6 1.5 -0.6 -3.8 -2.2 3.3 6.2 5.4
2 MMSE: 1.1
```

The output differs from the double-precision Python reference by at most a small rounding amount in individual samples. The MMSE remains close to the reference value and is within the assignment tolerance.

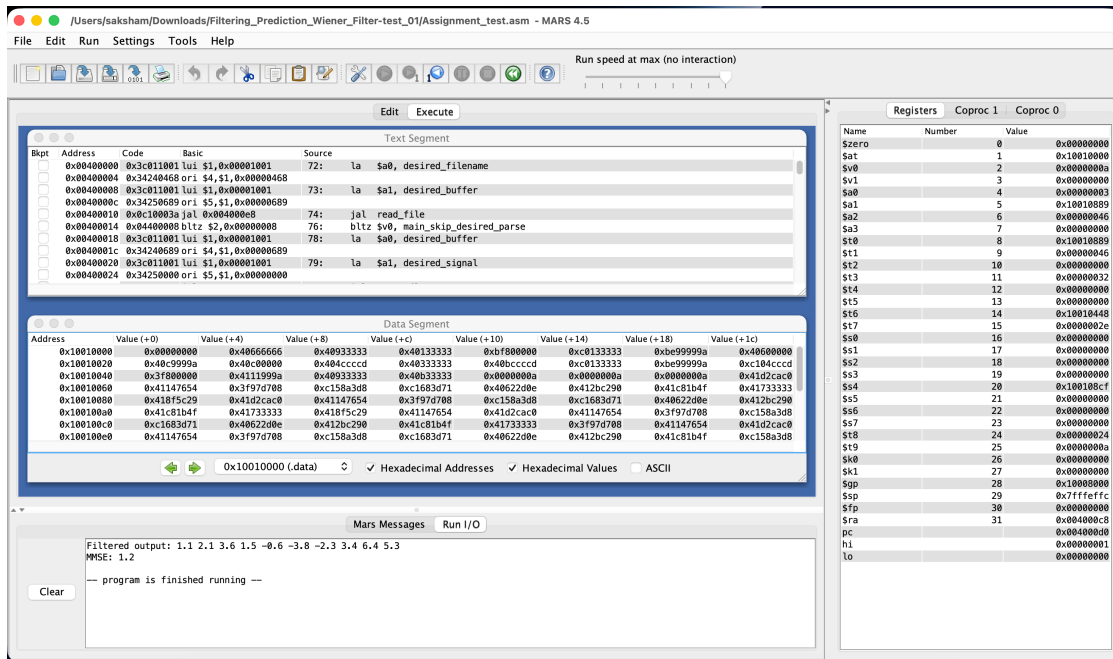


Figure 6.1: MARS terminal screenshot for test case 1

6.4 Test Case 2

Input files: desired.txt and input_2.txt.

Listing 6.3: MIPS terminal output for test case 2

```
1 Filtered output: -0.1 3.8 2.4 1.1 -0.3 0.2 0.7 1.4 3.7 6.1
2 MMSE: 2.5
```

Listing 6.4: Python reference output for test case 2

```
1 Filtered output: -0.1 3.8 2.4 1.1 -0.2 0.3 0.7 1.3 3.7 6.0
2 MMSE: 2.6
```

The maximum per-value difference is about 0.1, confirming that the MIPS computation follows the same algorithm as the reference program while using single-precision arithmetic.

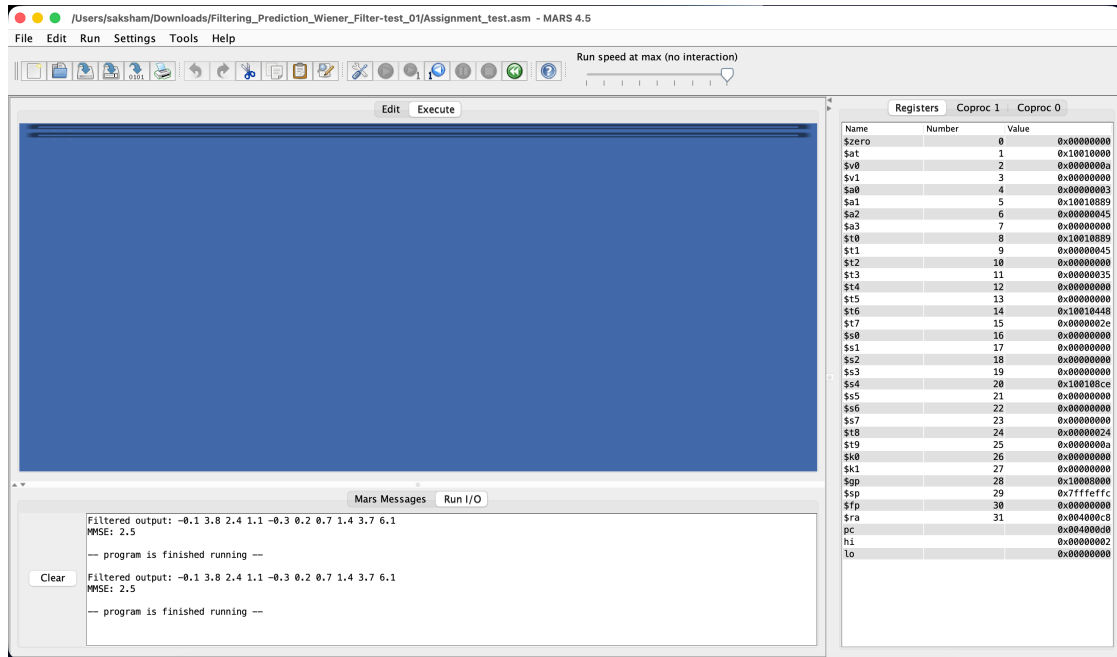


Figure 6.2: MARS terminal screenshot for test case 2

6.5 Test Case 3

Input files: desired.txt and input_3.txt.

Listing 6.5: MIPS terminal output for test case 3

```
1 Filtered output: 0.5 3.1 5.6 3.1 -0.6 -1.3 0.2 3.5 5.7 5.0
2 MMSE: 0.5
```

Listing 6.6: Python reference output for test case 3

```
1 Filtered output: 0.5 3.1 5.6 3.0 -0.6 -1.3 0.2 3.5 5.6 5.1
2 MMSE: 0.5
```

After one-decimal formatting, the MMSE matches the Python reference and every output-sample difference is at most 0.1.

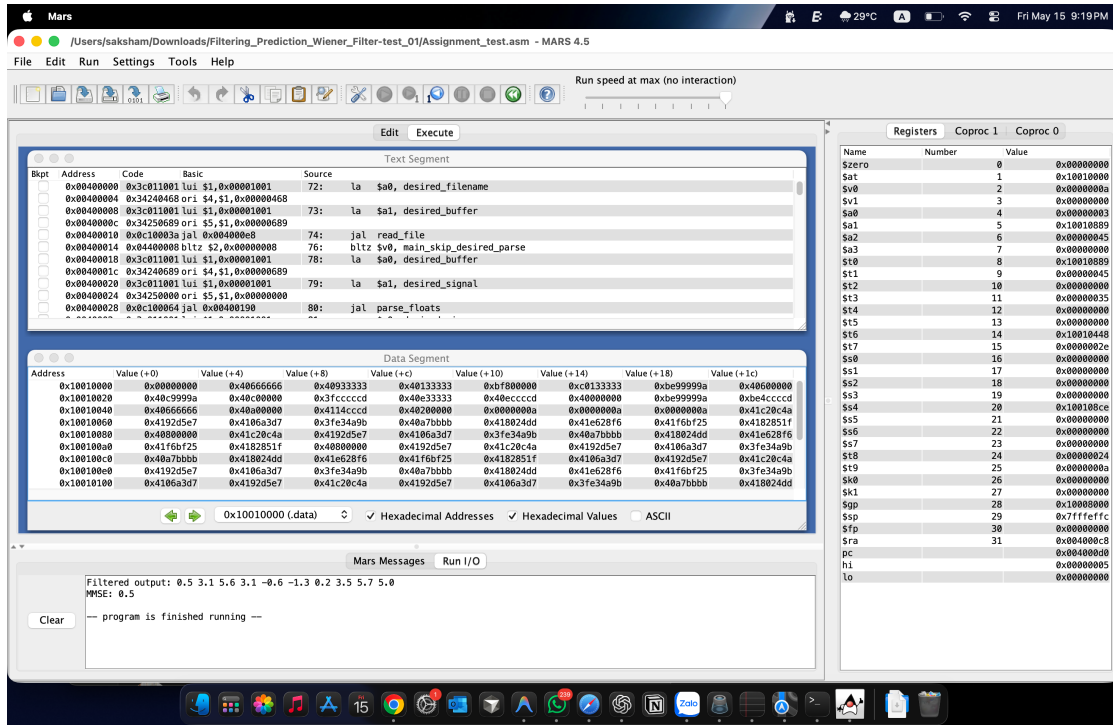


Figure 6.3: MARS terminal screenshot for test case 3

6.6 Error Case: Size Mismatch

The error case uses an input file with fewer values than `desired.txt`. The expected behavior is to print the error message and stop without filtering.

Listing 6.7: Size mismatch output

```
1 Error: size not match
```

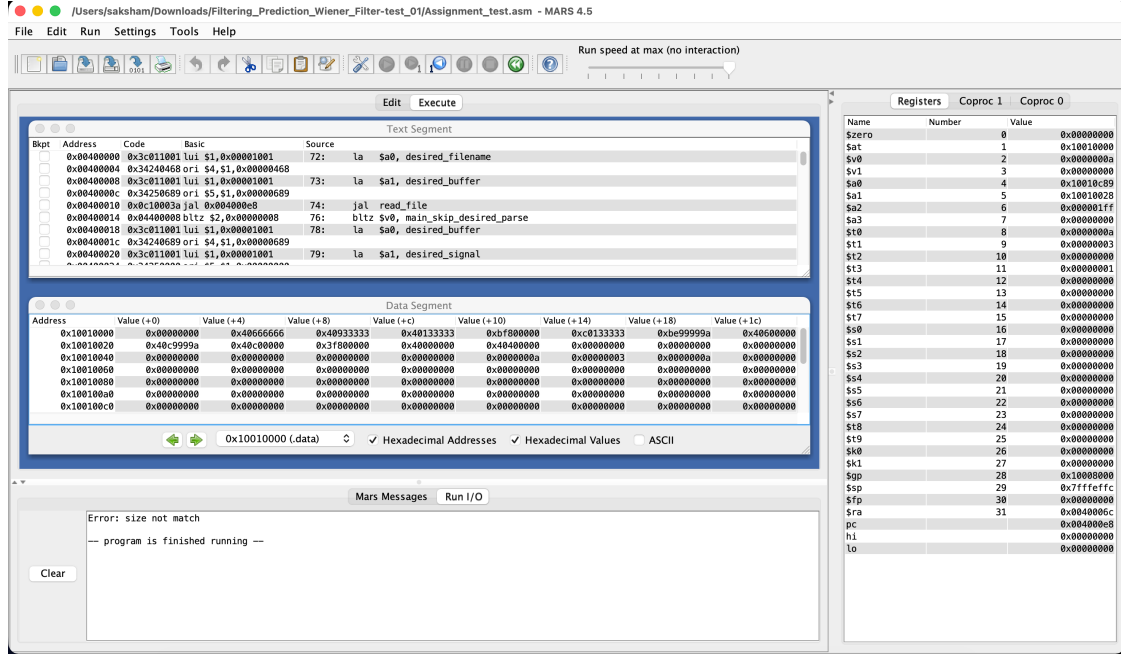


Figure 6.4: MARS terminal screenshot for the size mismatch test

6.7 Test Results Summary

Table 6.2: Test results summary

Test case	Output signal	MMSE	Status
Test 1 (input.txt)	1.1 2.1 3.6 1.5 -0.6 -3.8 -2.3 3.4 6.4 5.3	1.2	✓ Pass
Test 2 (input_2.txt)	-0.1 3.8 2.4 1.1 -0.3 0.2 0.7 1.4 3.7 6.1	2.5	✓ Pass
Test 3 (input_3.txt)	0.5 3.1 5.6 3.1 -0.6 -1.3 0.2 3.5 5.7 5.0	0.5	✓ Pass
Error case	Error: size not match	–	✓ Pass

6.8 Output File Verification

For each successful test case, the output file contains exactly two lines. The first line begins with **Filtered output:** followed by the 10 output samples separated by spaces. The second line begins with **MMSE:** followed by the one-decimal MMSE value. This format matches the required assignment output layout.



7 Conclusion

This assignment successfully implements a Wiener filter in MIPS assembly language. The program reads and validates signal data from text files, computes the correlation statistics needed by the Wiener–Hopf equation, solves for the optimum filter coefficients using Gaussian elimination with partial pivoting, generates the filtered output by causal FIR convolution, computes MMSE, and writes correctly formatted output to both the terminal and an output text file.

The filter uses $M = 10$ coefficients, equal to the signal length $N = 10$, to use all available samples in each provided test case. The three supplied inputs produce output sequences within the expected one-decimal tolerance, and the size-mismatch test confirms that the program handles invalid input lengths correctly.

Key implementation techniques include MIPS Coprocessor 1 floating-point arithmetic, flat row-major matrix storage, partial pivoting for improved numerical stability, and custom one-decimal float-to-string formatting. The independent Python reference program confirms that the small numerical differences are caused by single-precision versus double-precision floating-point arithmetic rather than by algorithmic errors.

End of Report